
When Should Small Games Use AI?

Ruangyot Nanchiang
rayato159@gmail.com

Abstract

This paper asks a practical question for small-scale game development: when is it worth replacing handcrafted decision rules with tiny learned policies? We study this tradeoff in a controlled real-time 2D action-game simulation where navigation, collision, cooldowns, animation, and melee execution remain rule-based; the policy only chooses one of five high-level tactical actions. We compare a handcrafted rule expert, an 8k MLP Q-network (7,980 actual parameters), and a 10k grouped-attention Q-network (9,325 actual parameters), selected from a 1k–10k rounded capacity sweep. The main result is deliberately not “AI beats rules.” The rule expert remains the strongest controller (5.76 ± 1.64 seconds timeout-penalized time to player defeat). The best learned policy is the 8k MLP (5.90 ± 0.31 seconds), and the best attention policy is the 10k grouped-attention model (6.18 ± 0.35 seconds). Attention raises the scalar action-diversity diagnostic over the selected MLP, but does not improve the primary speed metric. The study contributes a compact 23-feature enemy-agent state, 15-feature player-bot state, safe randomized spawn protocol, Rust Q-policy runtime, off-line DDQN-style trainer, three-seed evaluation protocol, and practical recommendations for deciding when tiny AI is worth its engineering cost in small games.

1 Introduction

Small games usually start with handcrafted AI for good reasons. Rules are deterministic, cheap, interpretable, easy to debug, and easy to ship. A designer can inspect the condition that fired, tune a cooldown, or remove a bad branch without retraining a model. For small teams, this matters as much as raw behavior quality: the fastest AI system is often the one the developer can understand at 2 a.m. before a build.

Neural game AI is attractive for the opposite reason. As decision complexity grows, handcrafted rules can turn into a brittle tree of exceptions: one branch for player stamina, another for weapon state, another for side pressure, another for spacing, another for the surrounding formation. The problem is not that any single rule is hard; the problem is that interactions between rules grow faster than the rule tree stays readable. The practical question is therefore not *is AI better than rules?* but *at what point does a tiny learned policy become worth the data, training, runtime, and debugging cost?*

This paper studies that question in a real-time Godot-based 2D action-game simulation rather than in a toy grid world. The environment is intentionally small and controlled: multiple enemy agents pressure a player in a closed arena, while the engine still owns navigation, collision, animation, cooldowns, and combat resolution. This setting is practically relevant because Godot has become an increasingly visible engine in indie-game development rather than a niche prototype target [1].

The central research question is deliberately practical:

When should a small-scale action game keep handcrafted tactical rules, and when do tiny learned Q-policies become a reasonable replacement for part of that decision logic?

The paper does not claim a general game intelligence system. The learned policy is only a tactical selector: it reads a compact state vector and chooses a high-level action. Low-level game systems remain deterministic and inspectable. This keeps the comparison close to how many shipped games are actually built: the reliable engine-side systems stay in control, while a small model attempts to replace part of the tactical if-else layer.

A secondary architecture question remains: under the same state/action interface and strict parameter ceiling, does grouped attention use player orientation and multi-agent geometry more effectively than a flat MLP? This is treated as a model-family comparison, not as a promise that attention is always worth the cost.

The answer should be read carefully. In the current benchmark, the compact rule expert is still the strongest default controller. The best learned policy is an 8k MLP, and the best attention model is a 10k grouped-attention policy. Attention changes action usage and raises the scalar diversity diagnostic over the selected MLP, but the primary speed score still favors direct pressure. For this small game, AI is not automatically a win; it becomes interesting only when the behavior surface is complex enough that maintaining rules becomes the more expensive path.

Our main contributions are as follows:

- We define a controlled small-scale action-game simulation for comparing handcrafted and learned tactical decision policies while keeping pathfinding, collision, cooldowns, and combat execution in the engine.
- We compare a rule expert, a tiny MLP Q-policy, and a tiny grouped-attention Q-policy under the same 23-feature state and five-action interface.
- We report three-seed final-model results, 1k–10k capacity sweeps, shaped return diagnostics, speed-score tables, and action-diversity plots generated from gameplay logs.
- We give practical guidance for when learned policies are worth using in a small game and when handcrafted rules remain the better engineering choice.

2 Related Work

Q-learning and deep Q-networks. Q-learning provides the off-policy temporal-difference target used in this work [3]. Deep Q-networks showed that neural value functions can learn control policies from high-dimensional observations [4], while Double-DQN reduces overestimation by selecting the next action with the online network and evaluating it with a target network [5]. Our trainer follows that Double-DQN target structure, but the deployed policy remains tiny and game-loop friendly.

Attention under strict budgets. Transformers introduced content-based attention as a general sequence mechanism [6]. This paper does not use a language-model decoder or predict sequences. Instead, it tests whether attention over a fixed set of feature tokens improves tactical Q-value estimation. Grouped-query attention shares key/value projections across query heads to reduce parameters [7]. The grouped-attention policy here adopts that parameter sharing under a 10k-parameter ceiling.

Runtime efficiency. Attention is useful only if the runtime cost fits the game loop. The Rust runtime therefore uses a streamed attention recurrence inspired by FLASH-D [8].



Figure 1: Actual Godot benchmark scene used for evaluation. The arena contains the player, a three-agent enemy group, obstacles, pickups, collision geometry, A* navigation, and a four-anchor safe spawn region. The learned policy sees only a compact tactical state; movement, collision, cooldowns, and combat resolution remain in the game engine.

This is not a GPU FlashAttention claim; it is a scalar Rust implementation that avoids materializing the full score vector for the tiny feature-token policy.

3 Simulation Environment and Tactical Interface

3.1 Simulation Environment

The environment is a simplified real-time 2D action-game simulation built in Godot. The player has eight-direction movement, armed and unarmed states, a two-step melee combo, stamina, health, and a roll/dodge action. The enemy group uses A* navigation, melee attacks, dash movement, stagger on hit, and map collision. Evaluation uses three enemy agents per episode for a fair fixed-count comparison. Player and enemy starts are sampled inside a four-marker safe quadrilateral and validated against navigation and collision before each episode, avoiding fixed-spawn bias. The learned policy is deployed as a Rust GDExtension through `godot-rust/gdext`, keeping the model runtime inside the same Godot 4 execution path as the game logic [2].

The simulation is intentionally abstracted away from a single game theme. It is not meant to prove that one commercial enemy type needs neural control. It is a controlled testbed for a common small-game decision problem: several simple agents must decide whether to commit, flank, retreat, or attack while the engine continues to handle the physical execution. State observations are collected from repeated autoplay runs, actions are generated through the same five-action interface for every controller, and evaluation freezes policies so the rule and learned systems face the same scenario distribution.

4 State and Reward

The state vector currently has 23 features. It is intentionally small enough that all tested policies stay below 10k parameters.

Action	Rule-based execution
ATTACK	Commit to melee pressure when feasible.
FLANK_LEFT	Move to a left-side cutoff slot around the player.
FLANK_RIGHT	Move to a right-side cutoff slot around the player.
DASH_IN	Burst toward attack range, then resume normal chasing.
DASH_OUT	Retreat briefly to bait stamina/weapon timing.

Table 1: Fixed tactical action space. The policy chooses an action; the game rules decide whether that action is physically feasible.



Figure 2: State/action visualization from the real Godot scene. The action badges show the current tactical outputs around the player, while the debug panel shows the compact model state used by the enemy policy. This figure connects the fixed five-action interface in Table 1 to the actual arena geometry: agents can pressure from the front, flank from the side, or dash away while the rule layer handles movement feasibility.

The enemy transition reward is shaped around tactical pressure: damage to the player, forced rolls, stamina reduction, cornering, surrounding, and player defeat are positive; enemy agents being hit or defeated, wasted dashes, and unproductive repeated direct attacks are negative. This shaped return is a training signal and a diagnostic, not the leaderboard objective. The primary evaluation objective is how quickly the enemy controller defeats the player inside the 60-second episode cap:

$$J_{\text{TTK}} = \mathbb{E}[\min(T_{\text{defeat}}, T_{\text{max}})], \quad T_{\text{max}} = 60 \text{ s},$$

where lower is better and failed player defeats are scored as the full timeout. For a bounded positive presentation of the same objective, the tables also report

$$S_{\text{speed}} = 1 - \frac{J_{\text{TTK}}}{T_{\text{max}}},$$

where higher is better. The shaped return is reported only as a learning and reward-design audit signal. A controller can therefore defeat the player quickly while receiving a poor shaped return if the dense reward over-penalizes the behavior that produces the defeat. This is a reward-shaping limitation, not a change in the evaluation objective.

Feature	Meaning
<code>is_player_armed</code>	Whether the player currently holds a weapon.
<code>is_player_stamina_less_half</code>	Player stamina gate.
<code>is_player_health_less_half</code>	Player health gate.
<code>is_player_stuck</code>	Player has made little movement recently.
<code>player_stuck_normal_x/y</code>	Direction away from enemy-group pressure.
<code>nearby_zombie_count</code>	Live enemy-agent count, clipped and normalized.
<code>dash_ready</code>	Dash action is tactically plausible by range.
<code>attack_ready</code>	Melee attack range gate.
<code>nearest_zombie_distance</code>	Normalized nearest-enemy distance.
<code>nearest_zombie_dir_x/y</code>	Direction from player to nearest enemy.
<code>player_speed</code>	Normalized player speed.
<code>swarm_centroid_dir_x/y</code>	Direction from player to enemy-group centroid.
<code>player_facing_dot</code>	Whether the nearest enemy is in front of or behind the player.
<code>nearest_zombie</code>	Left/right side of the nearest enemy relative to player facing.
<code>nearest_zombie_side_sign</code>	
<code>backstab_opportunity</code>	Normalized behind-player attack opportunity.
<code>swarm_left/right_pressure</code>	Fraction of enemy agents occupying each side of the player.
<code>swarm_front/back_pressure</code>	Fraction of enemy agents in front of or behind the player.
<code>swarm_spread</code>	Mean spread around the enemy-group centroid.

Table 2: Enemy-agent state schema consumed by the MLP and grouped-attention policies. Code-level feature names retain their original `zombie` prefix for checkpoint compatibility.

5 Model Architecture and Rule Expert

5.1 Rule Expert

The rule controller is intentionally small. It exists to answer the paper’s core question: how far can a simple hand-written tactical loop scale before a tiny Q-policy becomes worth its cost? At each sampling step it reads the same state buffer as the neural policies, but it uses only two tactical decisions. If an enemy can attack, the expert selects `ATTACK`; otherwise, if dash is ready, it selects `DASH_IN`; otherwise it falls back to `ATTACK`, which makes the existing A* pathfinding and melee execution keep moving the enemy group toward the player.

The rule expert never emits `FLANK_LEFT`, `FLANK_RIGHT`, or `DASH_OUT`. That restriction is deliberate. Flank choice, baiting, and front/side/back role coordination are the behaviors the learned policy is asked to learn or expose through its action values. This keeps the baseline readable: it is A* pursuit plus dash-in pressure plus attack timing, not a hidden hand-authored tactical tree disguised as a baseline.

5.2 Tiny MLP Q-Network

The MLP family has 23 inputs, one ReLU hidden layer, and five Q-value outputs. The selected learned controller uses hidden width $h = 275$, giving 7,980 runtime parameters. Smaller and larger widths appear only in the capacity sweep. During gameplay, the network is used only for action selection and transition logging. Given a 23-entry state vector s , normalization statistics μ, σ , hidden width h , and five actions, its forward pass is

$$x_j = \frac{s_j - \mu_j}{\sigma_j + \epsilon}, \quad z = \text{ReLU}(W_1 x + b_1),$$

$$Q_\theta(s, \cdot) = W_2 z + b_2.$$

Action selection uses ϵ -greedy exploration during collection and $\arg \max_a Q_\theta(s, a)$ during frozen evaluation. Weight updates are performed after episode collection. The intended trainer uses a Double-DQN target:

$$a_{t+1}^* = \arg \max_{a'} Q_\theta(s_{t+1}, a')$$

$$y_t = \begin{cases} r_t, & d_t = 1, \\ r_t + \gamma Q_{\theta^-}(s_{t+1}, a_{t+1}^*), & d_t = 0. \end{cases}$$

The deployed policy still contains only the primary policy network; the target network is a training-time artifact and is not counted in the runtime parameter budget.

5.3 Tiny Grouped-Attention Q-Network

The attention policy converts each scalar feature into a feature token using a learned feature embedding plus a shared value embedding. A learned Q token attends over all feature tokens. The selected attention controller uses two attention layers, model width 40, four query heads, and one shared key/value head, giving 9,325 runtime parameters. This is grouped-query attention: query heads are separate, but K/V projections are shared by groups to reduce parameters [7]. The attention block has no feed-forward sub-layer. In this setting, attention is used narrowly as a compact feature-relation module: it lets the Q token compare player gates, orientation, and multi-agent geometry before producing action values.

For feature i , scalar input x_i , learned feature embedding e_i , shared value embedding v , and learned query token q_0 , the token sequence is

$$t_i = e_i + x_i v, \quad T^{(0)} = [t_1; \dots; t_{23}; q_0].$$

Each attention layer applies RMS normalization before grouped attention:

$$\hat{T}^{(\ell)} = \text{RMSNorm}(T^{(\ell)}).$$

$$Q_h = \hat{T}^{(\ell)} W_h^Q, \quad K_g = \hat{T}^{(\ell)} W_g^K, \quad V_g = \hat{T}^{(\ell)} W_g^V.$$

Multiple query heads share fewer key/value heads by mapping query head h to KV group $g(h)$. The per-head output is

$$A_h = \text{Attn}(Q_h, K_{g(h)}, V_{g(h)}).$$

$$T^{(\ell+1)} = T^{(\ell)} + [A_1; \dots; A_H] W^O.$$

Only the final query token is read out:

$$r = \text{RMSNorm}(T_{24}^{(L)}), \quad Q_\theta(s, \cdot) = W_Q r + b_Q.$$

The Rust runtime now evaluates grouped attention with a FLASH-D-style streamed recurrence: for each query head, it updates the output using $w_i = \sigma(s_i - s_{i-1} + \log w_{i-1})$, where s_i is the current attention score. This hides the softmax division inside the non-linear weight recurrence and avoids materializing a full score vector [8]. The current implementation is scalar Rust inside Godot rather than a hardware accelerator, but the attention path is no longer the naive “store all scores, then softmax” form.

Budget	MLP hidden	MLP params	Grouped-attention config
1k	34	991	$d = 16$, 1 layer, 4 heads, 1 KV head
2k	68	1,977	$d = 24$, 1 layer, 4 heads, 1 KV head
4k	137	3,978	$d = 32$, 1 layer, 4 heads, 1 KV head
8k	275	7,980	$d = 36$, 2 layers, 4 heads, 1 KV head
10k	344	9,981	$d = 40$, 2 layers, 4 heads, 1 KV head

Table 3: Capacity sweep settings. Each budget is run with three seed repeats and reported as mean $\pm$ sample standard deviation.

6 Capacity Sweep

The final evaluation sweeps the policy capacity under the same fixed 23-feature, five-action interface. The architecture family is held fixed while parameter count changes. For the MLP, the hidden width is adjusted directly. For grouped attention, the sweep changes model width and layer count while keeping grouped query attention with four query heads and one shared key/value head where possible. All settings stay below or approximately at the 10k-parameter ceiling. The labels 1k, 2k, 4k, 8k, and 10k are rounded budget buckets, not exact parameter counts. Table 3 reports the sweep configuration, while Table 5 reports the exact post-training parameter count used in each evaluated row.

7 Collection, Training, and Evaluation

Both neural policies collect bot-only self-play transitions for 100 episodes. The initial enemy behavior is intentionally simple: navigate to the player, attack in range, and dash in when dash is ready. Exploration can still sample the full five-action interface, and the role dispatcher expands group-level policy actions into per-agent roles for three-enemy fights. For example, a group **ATTACK** decision maps the nearest enemy to front pressure while the next two agents fill left and right flanks; **DASH_OUT** maps one agent to bait, one to side pressure, and one to dash-in follow-up. Dash-in execution aims at a behind-player tactical slot when possible, and the state exposes left/right/front/back group occupancy so learned policies can coordinate formation rather than choosing actions as isolated agents.

Player attacks are directional: an enemy must lie in the player’s forward melee cone to take damage. This makes front/back positioning a real tactical variable instead of cosmetic state. The default training opponent is a player-side MLP Q-policy wrapped by a duelist controller, not a runaway survival bot: it is rewarded for hitting and defeating enemies, penalized for death, and biased toward armed engagement rather than corner camping. No human rows are used.

Collection uses $\epsilon = 1.0$, $\epsilon_{\min} = 0.1$, and per-episode decay 0.95. Each episode is capped at 60 seconds. Evaluation freezes neural checkpoints, fixes the enemy count to three, and runs 20 episodes per mode. The main score is the mean time to player defeat with timeouts counted as the full 60 seconds. Lower is better because it means the enemy group defeats the player faster and more reliably. Fastest successful defeat time, raw defeat count, defeat rate, reward, damage, and action diversity remain secondary diagnostics. Action diversity is normalized Shannon entropy over the actions that a controller actually emits in that evaluation file,

$$\mathcal{A}_m = \{a \mid c_m(a) > 0\}, \quad D_{\text{act}} = \frac{-\sum_{a \in \mathcal{A}_m} p(a) \log p(a)}{\log |\mathcal{A}_m|},$$

so $D_{\text{act}} = 0$ means one action dominates and $D_{\text{act}} = 1$ means the controller uses its emitted action set uniformly. This metric is diagnostic rather than a target: a two-action rule

Mode	Seeds	TTK/timeout (s)	Death rate	Fastest kill (s)	Speed score	Diversity
Rule	3	5.76 ± 1.64	0.98 ± 0.03	3.03 ± 0.34	0.90 ± 0.03	0.84 ± 0.08
MLP 8k	3	5.90 ± 0.31	1.00 ± 0.00	3.55 ± 0.31	0.90 ± 0.01	0.56 ± 0.02
Attention 10k	3	6.18 ± 0.35	1.00 ± 0.00	3.77 ± 0.16	0.90 ± 0.01	0.66 ± 0.10

Table 4: Final selected-policy evaluation after directional player melee. Lower TTK/timeout is better; speed score is the same timeout-penalized objective shown on a 0–1 scale where higher is better. The neural rows are the selected 8k MLP and 10k grouped-attention policies from Table 5.

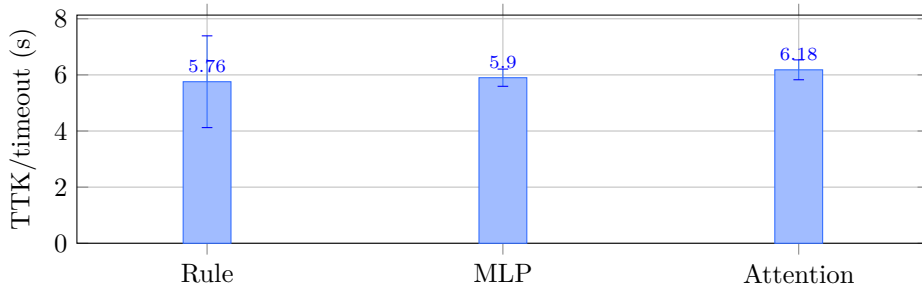


Figure 3: Final selected-policy primary score. Failed player defeats count as the full 60-second timeout, so lower is better.

baseline can score high if it balances those two actions, while a learned five-action policy can score lower if most states collapse toward one tactic. The serious evaluation protocol repeats the full collect–train–evaluate pipeline over three seeds and reports mean $\pm$ sample standard deviation. Any generated artifacts from earlier state schemas or checkpoint formats are excluded from the paper body because they are not comparable with the active protocol.

8 Results

All reported results in this section were regenerated after directional player melee, aggressive player-duelist rewards, and coordinated enemy role dispatch were enforced. Each neural policy collects 100 bot-only training episodes, is trained offline, and is then frozen for 20 evaluation episodes. The final comparison uses the selected rows from the capacity sweep: the 8k MLP and the 10k grouped-attention policy. Older 1k/2k default checkpoints are not used in the final comparison.

Under this unified accounting, the rule expert gives the best primary score at 5.76 ± 1.64 seconds TTK/timeout. The selected 8k MLP is close at 5.90 ± 0.31 seconds and defeats the player in all evaluation episodes. The selected 10k grouped-attention policy is slower at 6.18 ± 0.35 seconds but also defeats the player in all evaluation episodes. The result is deliberately not written as an AI victory: in this small arena, a compact direct-pressure rule remains the strongest tactic.

The capacity sweep adds model-size context. The best TTK/timeout in Table 5 is the 8k MLP at 5.90 ± 0.31 seconds, which is close to the rule expert but does not clearly beat it. The best attention point is the 10k grouped-attention model at 6.18 ± 0.35 seconds. The sweep is not monotonic: larger policies can become slower or less reliable when they overuse evasive or flanking actions instead of committing to the defeat. This supports the practical claim that tiny neural policies can become reasonable when rule branching grows, while also showing that learned policies are an engineering tradeoff rather than a guaranteed upgrade.

Figure 4 should be interpreted as tactical coverage, not as a standalone quality score. The rule expert mainly emits `ATTACK` and `DASH_IN`, which is enough to defeat the player quickly

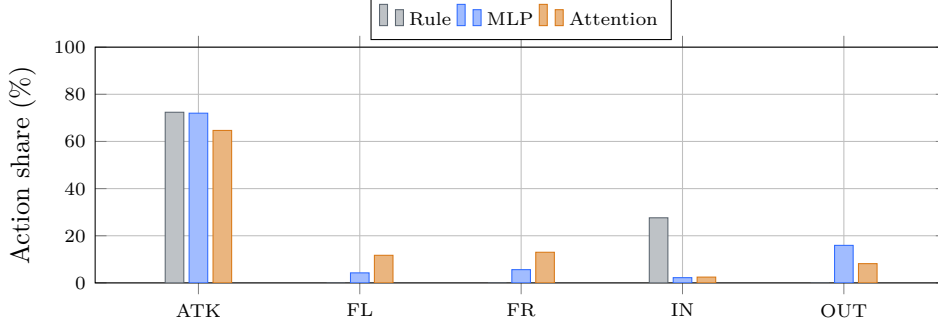


Figure 4: Final selected-policy action histogram averaged over three 20-episode evaluation seeds. Labels abbreviate **ATTACK**, **FLANK_LEFT**, **FLANK_RIGHT**, **DASH_IN**, and **DASH_OUT**. Rule expert uses only direct pressure actions, while learned policies can emit all five actions through the same feasibility layer.

Model	Budget	Params	Death rate	TTK/timeout (s)	Speed score
MLP	1k	991	1.00±0.00	6.09±0.19	0.90±0.00
MLP	2k	1977	0.98±0.03	7.30±1.11	0.88±0.02
MLP	4k	3978	1.00±0.00	6.08±0.27	0.90±0.00
MLP	8k	7980	1.00±0.00	5.90±0.31	0.90±0.01
MLP	10k	9981	1.00±0.00	6.78±0.92	0.89±0.02
Attention	1k	1157	0.97±0.03	8.65±0.96	0.86±0.02
Attention	2k	2213	0.98±0.03	7.92±2.54	0.87±0.04
Attention	4k	3589	1.00±0.00	6.85±1.13	0.89±0.02
Attention	8k	7673	1.00±0.00	6.62±0.53	0.89±0.01
Attention	10k	9325	1.00±0.00	6.18±0.35	0.90±0.01

Table 5: Capacity sweep over 1k–10k parameter budgets. The best learned primary score is the 8k MLP, but several tiny policies cluster near the same TTK/timeout. Speed score is derived from TTK/timeout; shaped reward is intentionally omitted from this ranking table and shown only as a diagnostic curve.

in the current arena. The learned policies can emit all five actions. In this representative run, the selected 10k attention policy uses more explicit flank actions than the selected 8k MLP, while the MLP uses more **DASH_OUT**. Both learned policies still emit **ATTACK** often because the primary objective rewards fast player defeats and the feasibility layer converts many close-range tactical decisions into attack opportunities. The important point is not that either histogram is automatically better, but that neural policies open actions the rule expert never emits. That extra variety does not improve the primary speed metric in the final comparison.

9 Discussion

9.1 Practical Recommendations for Small-Scale Game AI

For simple mechanics, small state spaces, and a small action set, handcrafted rules should remain the default choice. They have no training cost, no dataset pipeline, no checkpoint/runtime ABI, and no statistical failure mode. In this benchmark, the rule expert is not a toy baseline: it encodes direct A* pursuit, attack-at-range execution, dash-in pressure, spawn safety, and rule-based feasibility. Because the current arena rewards fast commitment, that compact rule is still the best controller.

Tiny learned policies become more practical when the decision surface grows faster than the

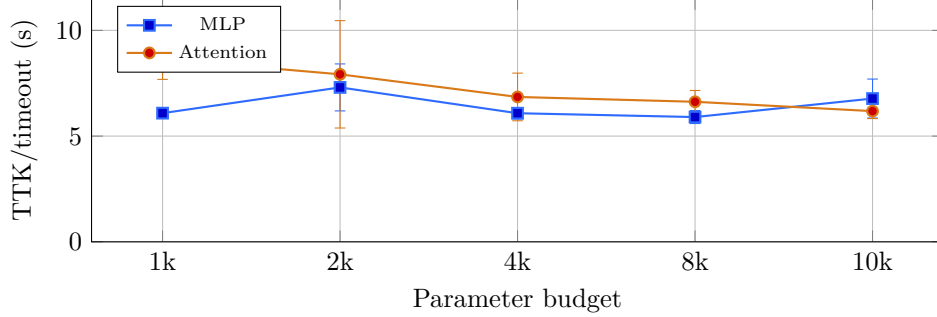


Figure 5: Capacity sweep primary score. The 8k MLP is the best learned point; attention improves at larger capacity but does not monotonically dominate MLP.

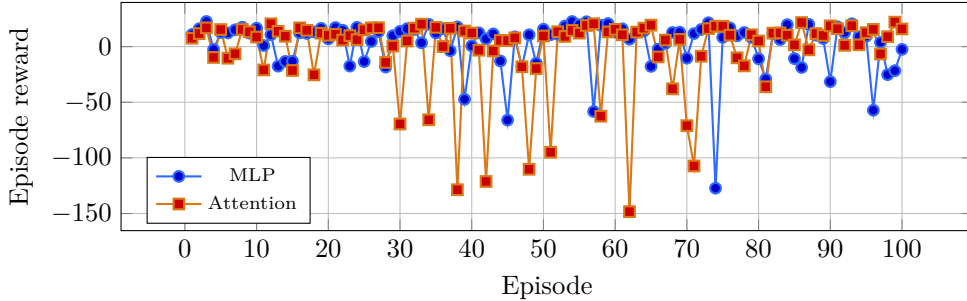


Figure 6: Representative transition-collection reward curves from the latest 100-episode training round. Rewards are shaped pressure signals used for training diagnostics, not the primary evaluation score.

rule tree can stay readable. This happens when behavior depends on several interacting gates such as player stamina, health, spacing, target side, formation occupancy, recent movement, and action cooldowns. In that setting, an 8k MLP can act as a compact replacement for part of the tactical branching layer while leaving navigation and combat execution deterministic. The benefit is not magic intelligence; the benefit is moving a growing set of interactions into a small function approximator that can be trained, frozen, and evaluated against fixed seeds.

Attention-based policies require a higher bar. They are most defensible when the decision depends on relations between agents or history-like context: front/side/back pressure, which agent is nearest, whether flanks are occupied, or whether retreat creates a later punish. If the game only needs a direct commit-or-attack tactic, grouped attention is overbuilt. In this study it increases tactical coverage but does not improve the primary speed metric.

9.2 Interpreting the Results

The rule expert remains an important comparison because it represents real designer labor rather than a deliberately weak baseline. It deliberately avoids learned flank roles and learned dash-out baiting. That baseline is still strong in a narrow three-enemy duel, which is exactly why the comparison is useful. The current learned policies do not clearly beat it on the primary score. The strongest learned result is instead a capacity result: an 8k MLP that matches the rule expert within the observed variance while keeping the same fixed tactical interface.

The grouped-attention result is mixed rather than theatrical. In the final action histogram, both neural policies emit flanks and DASH_OUT, while the rule expert remains locked to

ATTACK and **DASH_IN**. The 10k attention policy has higher scalar diversity and more explicit flank usage than the selected 8k MLP, but this does not improve the primary speed metric. The fixed three-enemy arena rewards fast commitment more than long tactical circling, so an MLP can beat the more expressive attention policy by choosing fewer, sharper actions.

Action diversity should be read alongside the histogram, not alone. The scalar diversity number is normalized Shannon entropy over the actions emitted by a controller. A rule controller that balances only **ATTACK** and **DASH_IN** can therefore score high even though it never emits flanks or dash-out. Conversely, a learned policy can score lower while still exposing all five actions if most states collapse toward one high-value tactic. The histogram is included to prevent that scalar from being over-interpreted. A controller can achieve high diversity while performing poorly, and it can achieve low diversity while repeatedly selecting an effective tactic.

The shaped-return mismatch is also intentional to report, not hide. This paper ranks controllers by timeout-penalized defeat speed; shaped return is a heuristic used to train and inspect the policy. Negative return for a fast controller means the dense reward still penalizes some behaviors that nevertheless defeat the player quickly. That is a reward-design limitation and a useful audit signal. The current conclusion is therefore modest: tiny neural Q-policies can compress tactical branching, but reward shaping still needs work if shaped return is expected to agree with the primary objective.

The previous draft reported Rust/Godot latency for older 1k/2k checkpoints. Those numbers are removed from the final comparison because they do not measure the selected 8k MLP and 10k grouped-attention policies. Runtime cost is therefore treated qualitatively here: grouped attention has a larger parameter count and a heavier forward path than the MLP, and final microsecond measurements should be reported only after a stable Godot/Rust benchmark is rerun on the selected checkpoints.

10 Limitations and Next Steps

This study is scoped to one Godot test map and one fixed tactical state ABI. Held-out map layouts, stronger player policies, and more seed repeats are needed before making a broad small-game AI claim. The action-diversity score should also be treated as a coarse diagnostic rather than proof of coordinated planning; future work should report state-conditioned diversity, surround completion, and flank-hit rates on held-out spawn layouts. The Rust runtime and Rust offline trainer use a Double-DQN-style target calculation. The grouped-attention trainer currently keeps the attention stack fixed and trains the Q readout, matching the lightweight runtime update path. We therefore do not claim full attention-stack backpropagation in the current implementation.

11 Code and Artifact Availability

The model-side Rust artifacts are released on GitHub.

`Rayato159/tiny-zombie-q-policies`

The repository is intended to contain the policy definitions, checkpoint format, CSV schemas, evaluation utilities, paper result tables, and demo media. The full game source is kept separate; the paper artifact focuses on the reproducible model-side contract used by the Godot/Rust runtime. The repository name is historical; the paper framing treats the environment as a general small-scale action-game simulation.

12 Conclusion

Tiny neural policies can run inside a Godot/Rust game loop under a strict parameter budget and can compete with a hand-written tactical rule expert on timeout-penalized defeat speed. Under unified metrics, the current winner is still the compact rule expert. The best learned policy is the 8k MLP from the capacity sweep, while the best attention policy is the 10k grouped-attention model. For this small game, AI is therefore not automatically worth using for raw performance alone.

The useful conclusion is conditional. Rules are still the right default when the desired behavior is narrow, direct, and easy to inspect. Tiny learned policies become worth considering when the number of interacting state/action cases makes the rule tree harder to maintain than a small training and evaluation pipeline. Attention is useful as an architectural probe because it changes action diversity through the same interface, but it is not justified by the primary speed metric in the present arena. The broader lesson is that architecture alone does not settle the decision: feature design, directional combat rules, reward shaping, role dispatch, capacity, runtime cost, and developer effort all matter.

References

- [1] J. Holfeld. On the relevance of the Godot Engine in the indie game development industry. *arXiv:2401.01909v2*, submitted 27 Dec. 2023, revised 10 Jan. 2024. doi:10.48550/arXiv.2401.01909.
- [2] The godot-rust project. The godot-rust book: User guide for gdxext, the Rust binding for Godot 4. <https://godot-rust.github.io/book/>. Accessed 2026-06-10.
- [3] C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [5] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with Double Q-learning. In *AAAI*, 2016. arXiv:1509.06461v3, submitted 22 Sept. 2015, revised 8 Dec. 2015. doi:10.48550/arXiv.1509.06461.
- [6] A. Vaswani, N. Shazeer, N. Parmar, et al. Attention is all you need. In *NeurIPS*, 2017. arXiv:1706.03762v7, submitted 12 June 2017, revised 2 Aug. 2023. doi:10.48550/arXiv.1706.03762.
- [7] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sang-hai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv:2305.13245v3*, submitted 22 May 2023, revised 23 Dec. 2023. doi:10.48550/arXiv.2305.13245.
- [8] K. Alexandridis, V. Titopoulos, and G. Dimitrakopoulos. FLASH-D: FlashAttention with hidden softmax division. *arXiv:2505.14201v1*, submitted 20 May 2025. IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED), 2025. doi:10.48550/arXiv.2505.14201.
- [9] C. W. Reynolds. Flocks, herds and schools: A distributed behavioral model. In *SIGGRAPH*, 1987.